QUEZON CITY UNIVERSITY
COLLEGE OF COMPUTER STUDIES

WEEKS 7-8
REQUIREMENTS MODELING: FLOW, BEHAVIOR, AND PATTERNS

SE101-SOFTWARE ENGINEERING

Topics:

- Requirements Engineering Concepts
- Requirements Modeling Strategies
- Flow-Oriented Modeling
- Creating a Behavioral Model
- Patterns for Requirements Modeling
- Metrics for Requirements Model

Requirements Engineering

- The process of establishing the services that the customer requires from a system and the constraints under which it operates and is developed.
- The requirements themselves are the descriptions of the system services and constraints that are generated during the requirements engineering process.

What is a Requirement?

- It may range from a high-level abstract statement of a service or of a system constraint to a detailed mathematical functional specification.
- This is inevitable as requirements may serve a dual function
  - May be the basis for a bid for a contract - therefore must be open to interpretation;
  - May be the basis for the contract itself - therefore must be defined in detail;
  - Both these statements may be called requirements.

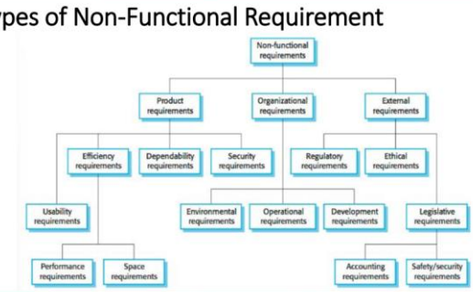
Types of Requirement

- **User requirements**
  - Statements in natural language plus diagrams of the services the system provides and its operational constraints. Written for customers.
- **Domain requirements**
  - Constraints on the system from the domain of operation
- **System requirements**
  - A structured document setting out detailed descriptions of the system's functions, services and operational constraints. Defines what should be implemented so may be part of a contract between client and contractor.
- **Non Functional Requirements**
- **Functional Requirements**

Functional and Non-Functional Requirements

FUNCTIONAL REQUIREMENTS	NON-FUNCTIONAL REQUIREMENTS
• Statements of services the system should provide, how the system should react to particular inputs and how the system should behave in particular situations. • May state what the system should not do. **Example 1:** The banking app should allow users to transfer money between accounts. **Example 2:** The e-commerce website should let users add items to a shopping cart and checkout.	• Constraints on the services or functions offered by the system such as timing constraints, constraints on the development process, standards, etc. • Often apply to the system as a whole rather than individual features or services. **Example 1:** The banking app must process transactions within 2 seconds. **Example 2:** The website should be available 99.9% of the time.

Types of Non-Functional Requirement



Non-Functional Classifications

- **Product requirements**
  - Requirements which specify that the delivered product must behave in a particular way e.g. execution speed, reliability, etc.
- **Organizational requirements**
  - Requirements which are a consequence of organisational policies and procedures e.g. process standards used, implementation requirements, etc.
- **External requirements**
  - Requirements which arise from factors which are external to the system and its development process e.g. interoperability requirements, legislative requirements, etc.

Metrics for Specifying Non-Functional Requirements

Property	Measure
Speed	Processed transactions/second User/event response time Screen refresh time
Size	Mbytes Number of ROM chips
Ease of use	Training time Number of help frames
Reliability	Mean time to failure Probability of unavailability Rate of failure occurrence Availability
Robustness	Time to restart after failure Percentage of events causing failure Probability of data corruption on failure
Portability	Percentage of target dependent statements Number of target systems

Non-Functional Requirements Implementation

- Non-functional requirements may affect the overall architecture of a system rather than the individual components.
  - For example, to ensure that performance requirements are met, you may have to organize the system to minimize communications between components.
- A single non-functional requirement, such as a security requirement, may generate a number of related functional requirements that define system services that are required.
  - It may also generate requirements that restrict existing requirements.

Requirements Imprecision

- Problems arise when requirements are not precisely stated.
- Ambiguous requirements may be interpreted in different ways by developers and users.
- Consider the term 'search' in requirement 1
  - User intention – search for a patient name across all appointments in all clinics;
  - Developer interpretation – search for a patient name in an individual clinic. User chooses clinic then search.

Requirements Completeness and Consistency

In principle, requirements should be both complete and consistent.

- Complete
  - They should include descriptions of all facilities required.
- Consistent
  - There should be no conflicts or contradictions in the descriptions of the system facilities.
- In practice, it is impossible to produce a complete and consistent requirements document.

Requirements should cover everything needed without contradictions, but in real life, perfection is almost impossible.

Goals and Requirements

- Non-functional requirements may be very difficult to state precisely and imprecise requirements may be difficult to verify.
- Goal
  - A general intention of the user such as ease of use.
- Verifiable non-functional requirement
  - A statement using some measure that can be objectively tested.
- Goals are helpful to developers as they convey the intentions of the system users.

Goals show what users want, but to build and test the system, goals need to be turned into measurable non-functional requirements.

- **Requirements Analysis**
  - Requirements analysis results in the specification of software's operational characteristics
  - It indicates software's interface with other system elements, and establishes constraints that software must meet.

Requirements analysis figures out what the software should do, how it connects to other systems, and what rules it must follow.

- **The Requirements Model**
  -A set of models; the first technical representation of a system.

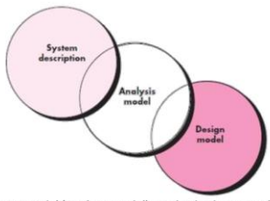


Fig. 7.1 The requirements model (Analysis model) as a bridge between the system description and the design model

Requirements Modeling Strategies

- **Structured Analysis** -considers data and the processes that transform the data as separate entities. Data objects are modeled in a way that defines their attributes and relationships. Processes that manipulate data objects are modeled in a manner that shows how they transform data as data objects flow through the system. **Structured Analysis = Data + Processes**
- **Object-oriented Analysis** -focuses on the definition of classes and the manner in which they collaborate with one another to effect customer requirements. UML and the Unified Process are predominantly object-oriented. **Object-Oriented Analysis = Objects + Their Interactions**

Requirements Modeling Strategies

EXAMPLE OF STRUCTURED ANALYSIS	EXAMPLE OF OBJECT ORIENTED ANALYSIS
Scenario: Hospital appointment system • **Data objects:** Patient, Appointment, Doctor • **Attributes:** Patient → Name, ID, Age Appointment → Date, Time, Patient ID, Doctor ID • **Processes:** Schedule Appointment → takes Patient and Doctor data, creates an Appointment Cancel Appointment → removes an Appointment from the system	**Scenario: Same hospital appointment system** **Classes:** Patient, Appointment, Doctor **Attributes (within each class):** Patient → Name, ID, Age Appointment → Date, Time Doctor → Name, ID, Specialty **Methods (behaviors within classes):** Patient → register(), updateInfo() Appointment → schedule(), cancel() Doctor → assignAppointment()

Requirements Modeling Strategies

The requirements modeling (or analysis modeling) action results in one or more of the following types of models:

- **Scenario-based models** of requirements from the point of view of various system "actors"
- **Data models** that depict the information domain for the problem
- **Class-oriented models** that represent object-oriented classes (attributes and operations) and the manner in which classes collaborate to achieve system requirements
- **Flow-oriented models** that represent the functional elements of the system and how they transform data as it moves through the system
- **Behavioral models** that depict how the software behaves as a consequence of external "events"

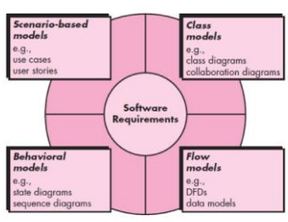


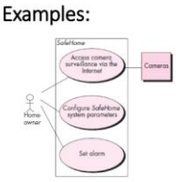
Fig. 7.2 Elements of the Requirements Model (or Analysis Model)

Scenario-based Modeling

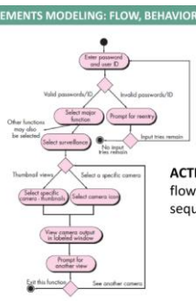
- If you understand how end users (and other actors) want to interact with a system, your software team will be better able to properly characterize requirements and build meaningful analysis and design.
- Requirements modeling with UML begins with the creation of scenarios in the form of **use cases, activity diagrams, and swimlane diagrams.**

Start by understanding user interactions, then model them visually using UML to guide analysis and design.

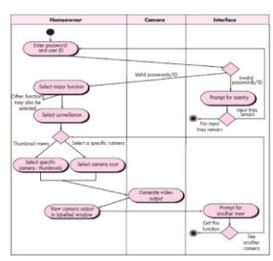
Examples:



USE CASE DIAGRAM - step-by-step stories of how users perform tasks.



ACTIVITY DIAGRAM – flowcharts showing the sequence of actions.



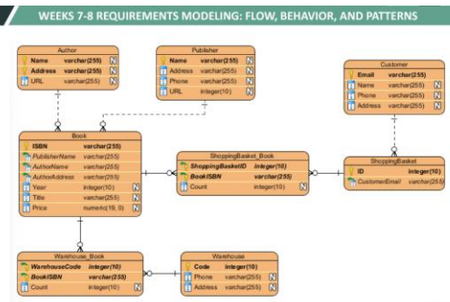
SWIMLANE DIAGRAM - activity diagrams that show who does what in each part of the system.

Data Modeling

- The Software Engineer or Analyst defines all data objects that are processed within the system, the relationships between the data objects, and other information that is pertinent to the relationships.
- **Entity-relationship diagram (ERD)** addresses these issues and represents all data objects that are entered, stored, transformed, and produced within an application.

EXAMPLE:

Entity-Relationship Diagram (ERD): - A visual tool that shows data objects (entities) and their relationships.

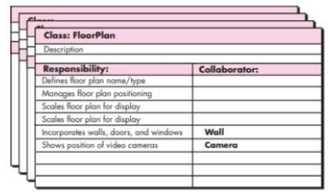


Class-based Modeling

- Class-based modeling represents the objects that the system will manipulate, the operations (also called methods or services) that will be applied to the objects to effect the manipulation, relationships (some hierarchical) between the objects, and the collaborations that occur between the classes that are defined.
- The elements of a class-based model include classes and objects, attributes, operations, class responsibility- collaborator (CRC) models, collaboration diagrams, and packages.

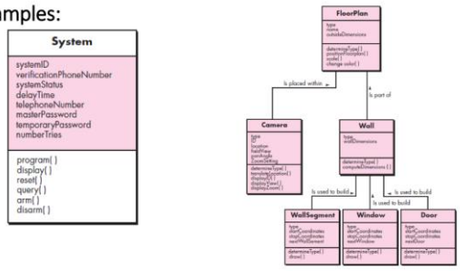
Class-based modeling is like designing a team of workers (classes) where each has a job (operations), keeps records (attributes), and works with others (relationships/collaborations).

Examples:



CRC (Class-Responsibility-Collaborator) models: Shows what each class is responsible for and which other classes it interacts with.

Examples:



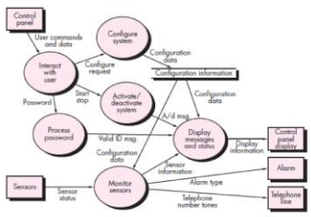
Flow-Oriented Modeling

- Data Flow Diagram (DFD) can be used to complement UML diagrams and provide additional insight into system requirements and flow.
- The DFD takes an input-process-output view of a system. That is, data objects flow into the software, are transformed by processing elements, and resultant data objects flow out of the software.
- The DFD is presented in a hierarchical fashion. That is, the first data flow model (sometimes called a level 0 DFD or context diagram) represents the system as a whole.

DFD is like a map showing how information flows through a factory: raw materials go in, get processed, and finished products come out.

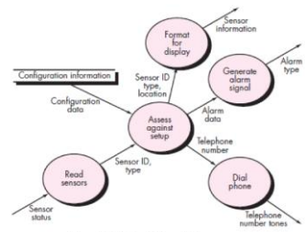
Examples:

Context Diagram (Level 0)



Level 1 Data Flow Diagram

Examples:



Level 2 Data Flow Diagram

Behavioral Modeling

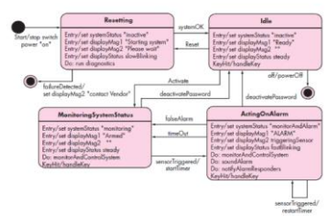
The behavioral model indicates how software will respond to external events or stimuli.

Creating a Behavioral Model

- To create the model, you should perform the following steps:
  1. Evaluate all use cases to fully understand the sequence of interaction within the system.
  2. Identify events that drive the interaction sequence and understand how these events relate to specific objects.
  3. Create a sequence for each use case.
  4. Build a state diagram for the system.
  5. Review the behavioral model to verify accuracy and consistency.

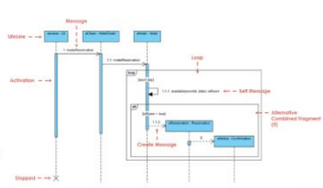
Behavioral Models

- **State Diagram**
  - In UML, it represents active states for each class and the events (triggers) that cause changes between these active states
  - State diagrams show how objects move between states based on events.



Behavioral Models

- **Sequence diagram**
  - It indicates how events cause transitions from object to object. Once events have been identified by examining a use case, the modeler creates a sequence diagram—a representation of how events cause flow from one object to another as a function of time.
  - Sequence diagrams show the flow of events between objects over time.



Patterns of Requirements Modeling

- Software patterns are a mechanism for capturing domain knowledge in a way that allows it to be reapplied when a new problem is encountered.

- **Discovering Patterns**
- The requirements model is comprised of a wide variety of elements: scenario-based (use cases), data-oriented (the data model), class-based, flow-oriented, and behavioral. Each of these elements examines the problem from a different perspective, and each provides an opportunity to discover patterns that may occur throughout an application domain, or by analogy, across different application domains.

Software patterns are reusable solutions discovered by studying system models from different perspectives.

- Metrics for Requirements Model

The Software Requirements Document

- The software requirements document is the official statement of what is required of the system developers.
- Should include both a definition of user requirements and a specification of the system requirements.
- It is NOT a design document. As far as possible, it should set of WHAT the system should do rather than HOW it should do it.

Agile Methods and Requirements

- Many agile methods argue that producing a requirements document is a waste of time as requirements change so quickly.
- The document is therefore always out of date.
- Methods such as XP use incremental requirements engineering and express requirements as **'user stories'.**
- This is practical for business systems but problematic for systems that require a lot of pre-delivery analysis (e.g. critical systems) or systems developed by several teams.

User Story

Title:	Priority:	Estimate:
User Story: As a [description of user], I want [functionality] so that [benefit].		
Acceptance Criteria: Given [how things begin] When [action taken] Then [outcome of taking action]		

Users of a Requirements Document

- **System customers/Business Units -** Specify the requirements and read them to check that they meet their needs. Customers specify changes to the requirements.
- **Managers** – use the requirements document to plan a bid for the system and to plan the system development process.
- **System Engineers** – use the requirements to understand what system is to be developed.
- **System Test Engineers** – use the requirements to develop validation tests for the system.
- **System Maintenance Engineers** – Use the requirements to understand the system and the relationship between its parts.

Requirements Document Variability

- Information in requirements document depends on type of system and the approach to development used.
- Systems developed incrementally will, typically, have less detail in the requirements document.
- Requirements documents standards have been designed e.g. IEEE standard. These are mostly applicable to the requirements for large systems engineering projects.

The Structure of a Requirements Document

Chapter	Description
Preface	This should define the expected readership of the document and describe its version history, including a rationale for the creation of a new version and a summary of the changes made in each version.
Introduction	This should describe the need for the system. It should briefly describe the system's functions and explain how it will work with other systems. It should also describe how the system fits into the overall business or strategic objectives of the organization commissioning the software.
Glossary	This should define the technical terms used in the document. You should not make assumptions about the experience or expertise of the reader.
User requirements definition	Here, you describe the services provided for the user. The nonfunctional system requirements should also be described in this section. This description may use natural language, diagrams, or other notations that are understandable to customers. Product and process standards that must be followed should be specified.
System architecture	This chapter should present a high-level overview of the anticipated system architecture, showing the distribution of functions across system modules. Architectural components that are reused should be highlighted.

The Structure of a Requirements Document

Chapter	Description
System requirements specification	This should describe the functional and nonfunctional requirements in more detail. If necessary, further detail may also be added to the nonfunctional requirements. Interfaces to other systems may be defined.
System models	This might include graphical system models showing the relationships between the system components and the system and its environment. Examples of possible models are object models, data-flow models, or semantic data models.
System evolution	This should describe the fundamental assumptions on which the system is based, and any anticipated changes due to hardware evolution, changing user needs, and so on. This section is useful for system designers as it may help them avoid design decisions that would constrain likely future changes to the system.
Appendices	These should provide detailed, specific information that is related to the application being developed; for example, hardware and database descriptions. Hardware requirements define the minimal and optimal configurations for the system. Database requirements define the logical organization of the data used by the system and the relationships between data.
Index	Several indexes to the document may be included. As well as a normal alphabetic index, there may be an index of diagrams, an index of functions, and so on.

Requirements Specification

The process of writing don the user and system requirements in a requirements document.

- **User requirements** have to be understandable by end-users and customers who do not have a technical background.
- **System requirements** are more detailed requirements and may include more technical information.
- The requirements may be part of a contract for the system development
  - It is therefore important that these are as complete as possible.

Ways of Writing a System Requirements Specification

Notation	Description
Natural language	The requirements are written using numbered sentences in natural language. Each sentence should express one requirement.
Structured natural language	The requirements are written in natural language on a standard form or template. Each field provides information about an aspect of the requirement.
Design description languages	This approach uses a language like a programming language, but with more abstract features to specify the requirements by defining an operational model of the system. This approach is now rarely used although it can be useful for interface specifications.
Graphical notations	Graphical models, supplemented by text annotations, are used to define the functional requirements for the system; UML use case and sequence diagrams are commonly used.
Mathematical specifications	These notations are based on mathematical concepts such as finite-state machines or sets. Although these unambiguous specifications can reduce the ambiguity in a requirements document, most customers don't understand a formal specification. They cannot check that it represents what they want and are reluctant to accept it as a system contract

Guidelines for Writing Requirements

- Invent a standard format and use it for all requirements.
- Use language in a consistent way. Use shall for mandatory requirements, should for desirable requirements.
- Use text highlighting to identify key parts of the requirement.
- Avoid the use of computer jargon.
- Include an explanation (rationale) of why a requirement is necessary.